Section 3.4 — Distributed File Systems & Object Storage Internals

Enhanced Study Notes (Architecture-Heavy Section)

> ***Study Directive:*** *Give extra weight to before/after architecture diagrams and production topology comparisons between named systems (e.g., GFS vs HDFS vs Ceph vs S3, replication vs erasure coding, eventual vs strong consistency).*

Executive Overview

Distributed storage systems underpin all cloud infrastructure. Understanding GFS/HDFS, Ceph, and S3 architectures reveals how companies like Google, Amazon, and Netflix store and serve exabytes of data reliably. The fundamental challenge: how do you make thousands of commodity disks behave like one infinitely large, highly reliable storage system?

---

1. Distributed File System Architecture

1.1 The Problem at Scale

Before (Single-Node Storage):

```
Application → [Single NAS / SAN] → Disk array

Limits:
  Storage:    Max ~1PB per array (very expensive)
  Throughput: Max ~10 Gbps (one network uplink)
  Reliability: RAID protects against disk failure, not server failure
  Cost:        $10,000+/TB for enterprise SAN

  Google 2003: Needs to store 20PB of crawled web data.
               Single server: impossible.
               Solution: Build GFS — use thousands of cheap servers.
```

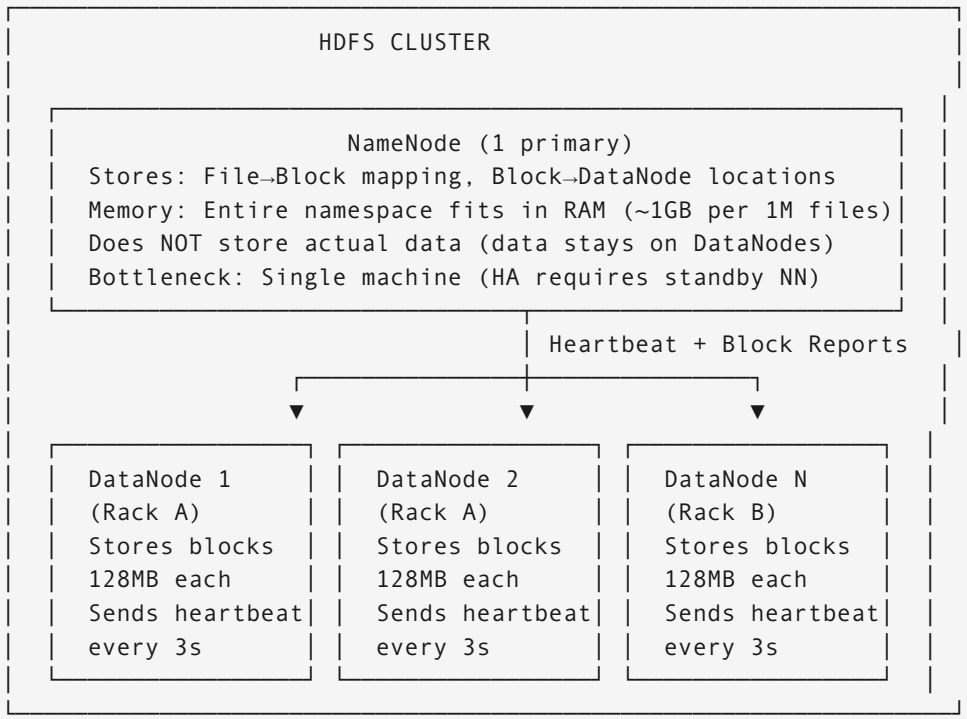
After (GFS / HDFS Distributed FS):

```
Application
    │
    ▼
NameNode (Metadata Server)
"Where is file X stored?"
    │
    ▼ Returns: [DataNode3:block1, DataNode7:block1, DataNode12:block1]
               [DataNode1:block2, DataNode5:block2, DataNode9:block2]
    │
    ▼
Client streams directly to/from DataNodes (bypasses NameNode for data!)
    │
    ├──► DataNode 3  (block 1, replica 1)
    ├──► DataNode 7  (block 1, replica 2)   ← Written in parallel
    └──► DataNode 12 (block 1, replica 3)

Scale:
  Google GFS: 1000 storage nodes = 1PB+ at commodity hardware cost
  Yahoo HDFS: 4000 nodes, 4PB in production (2011)
  Cost: ~$50/TB vs $10,000/TB for enterprise SAN
```

1.2 GFS / HDFS Architecture Deep Dive

Component Overview:



Block Placement Policy — Why It Matters:

```
File: video.mp4 (384MB) → 3 blocks of 128MB each

Naive placement (ALL on same rack):
  Block 1: DataNode1 (Rack A), DataNode2 (Rack A), DataNode3 (Rack A)

  Risk: Rack A top-of-rack switch fails → ALL 3 replicas unreachable!
        Entire file lost (temporarily or permanently if disks also fail)

HDFS Rack-Aware Placement:
  Block 1:
    Replica 1 → DataNode1 (Rack A) ← Write affinity: same rack as client
    Replica 2 → DataNode4 (Rack B) ← Different rack: tolerates rack failure
    Replica 3 → DataNode7 (Rack C) ← Different rack: 2nd datacenter if possible

  Result:
    Single disk failure:  2 replicas still available
    Single rack failure:  2 replicas still available (B + C)
    Entire datacenter A:  1 replica on Rack C still available
    Simultaneous 2-rack failure: 1 replica survives

Write throughput with rack-aware:
  Client → DataNode1 (Rack A) → pipeline to DataNode4 (Rack B)
                                          → pipeline to DataNode7 (Rack C)

  Pipeline: Each node forwards to next while receiving
            Client writes at full network speed
            No sequential round-trips needed
```

Client-Side Streaming (The Secret to HDFS Performance):

```
WRONG mental model: Client → NameNode → DataNode (NameNode proxies data)
CORRECT model:      Client → NameNode (metadata only) → Client → DataNode (data
direct)

Write path:
  1. Client: "I want to write /data/video.mp4"
  2. NameNode: "OK, use these DataNodes: [DN1, DN4, DN7]"
     (NameNode returns lease + DataNode list. Done with NameNode.)
  3. Client → DN1 → DN4 → DN7 (pipeline, no NameNode involved)
  4. DN7 ack → DN4 ack → DN1 ack → Client ack
  5. Client: "Write complete" → NameNode: "Update metadata"

Read path:
  1. Client: "Read /data/video.mp4, offset 256MB"
  2. NameNode: "Block 3 is on [DN3, DN8, DN12]. DN3 is closest to you."
  3. Client → DN3 directly (NameNode not involved)
  4. If DN3 fails during read: client retries to DN8
```

```
NameNode never touches data bytes → can serve millions of metadata ops/sec
without becoming bottleneck.
```

1.3 Ceph RADOS Architecture

Before Ceph (Directory-Based Mapping):

```
Central metadata server maps: object_id → storage_node
  object "a.jpg" → Node 3
  object "b.jpg" → Node 7

Problem: Metadata server = single point of failure + bottleneck
         Adding nodes requires updating central directory
         Millions of objects → huge metadata storage required
```

After Ceph CRUSH (Algorithmic Placement):

```
No central directory needed!
Any client can CALCULATE where any object is stored:
  hash(object_id, placement_group) → set of OSDs

CRUSH Map Hierarchy:
  Datacenter
    └── Rack A
          ├── Host 1
          │     ├── OSD 0 (Disk 0)
          │     └── OSD 1 (Disk 1)
          └── Host 2
                ├── OSD 2
                └── OSD 3
    └── Rack B
          └── Host 3
                ├── OSD 4
                └── OSD 5

Placement rule: "Store 3 replicas, one per rack"
CRUSH calculation for object "photo_abc":
  hash("photo_abc") % num_PGs = PG 42
  CRUSH(PG 42, rule) → [OSD 1 (Rack A), OSD 4 (Rack B), OSD 8 (Rack C)]
  Deterministic: any client gets same answer without any lookup!

Benefits:
  - No metadata server bottleneck
  - Adding nodes: update CRUSH map → CRUSH recalculates placement → rebalance
  - Client calculates placement: O(1) lookup, no network round-trip for metadata
```

Ceph Component Comparison:

Component	Role	Failure Impact
OSD (Object Storage Daemon)	Stores data, handles replication	1 OSD: normal, data re-replicated
MON (Monitor)	CRUSH map consensus (Paxos)	Need 3 of 5 for quorum
MDS (Metadata Server)	CephFS directory hierarchy	Only needed for CephFS, not RADOS
MGR (Manager)	Monitoring, stats, plugins	Not in data path

---

2. Object Storage Design (S3-Style)

2.1 S3 Internal Architecture

Simplified S3 Architecture:

```
Client PUT request:
  PUT /my-bucket/photos/vacation.jpg
       |
       ▼
  ┌──────────────────────────────────┐
  |  S3 Front-End (API Gateway)    |
  |  - Auth (IAM, signatures)      |
  |  - Bucket/key validation       |
  |  - Route to correct partition  |
  └──────────────┬─────────────────┘
                 | Route based on hash(bucket + key)
                 ▼
  ┌──────────────────────────────────┐
  |  Index Layer (Metadata)        |
  |  - Key → object location map   |
  |  - Version history             |
  |  - Access control              |
  └──────────────┬─────────────────┘
                 |
                 ▼
  ┌──────────────────────────────────┐
  |  Storage Layer                 |
  |  - Actual bytes                |
  |  - Erasure coded or replicated |
```

```
   |  - Checksummed (MD5/CRC)       |
   └────────────────────────────────┘
```

Multipart Upload — Numeric Example:

```
Problem: Upload 10GB file over unreliable network.
         Single PUT: connection drops at 9.9GB → restart from 0GB.

Solution: Multipart Upload

Step 1: Initiate
  POST /my-bucket/large-video.mp4?uploads
  Response: { "uploadId": "VXBsb2FkIElEIGZvciA2aWWpbmcncyBteS..." }

Step 2: Upload Parts (parallel)
  PUT /my-bucket/large-video.mp4?partNumber=1&uploadId=VXBs...
  Body: bytes 0–9,999,999 (10MB)
  Response: ETag: "d8e8fca2dc0f896fd7cb4cb0031ba249"

  PUT /my-bucket/large-video.mp4?partNumber=2&uploadId=VXBs...
  Body: bytes 10,000,000–19,999,999 (10MB)
  Response: ETag: "45b301d..."

  [Repeat for 1024 parts to cover 10GB at 10MB/part]
  [All parts upload in parallel: 10 concurrent = 10x speed]

Step 3: Complete
  POST /my-bucket/large-video.mp4?uploadId=VXBs...
  Body: [{PartNumber: 1, ETag: "d8e8..."}, {PartNumber: 2, ETag: "45b3..."}, ...]
  S3 assembles parts server-side, returns final ETag
  Response: ETag: "final_md5_of_all_parts-1024"

Resume example:
  Connection drops during part 500.
  Client: GET /my-bucket/large-video.mp4?uploadId=VXBs...&listParts
  Response: parts 1–499 completed.
  Client resumes from part 500.
  Only parts 500–1024 need re-upload.
  Saves: ~5GB of re-upload.
```

2.2 Consistency Models

Read-After-Write Consistency Problem:

```
Timeline without quorum:

t=0ms:  Client A: PUT /bucket/user-profile.json
        → Written to Storage Node 1 (primary)
t=1ms:  Client A receives: "200 OK, write successful"
t=2ms:  Replication to Node 2 and Node 3 begins (async)
t=5ms:  Client B: GET /bucket/user-profile.json
```

```
        → Routed to Node 2 (load balanced)
        → Node 2: hasn't received replication yet!
        → Returns: 404 Not Found (or stale version)

t=50ms: Replication completes. Now Node 2 has the object.
t=51ms: Client B: GET /bucket/user-profile.json → 200 OK, correct data.

Between t=2ms and t=50ms: Inconsistent window!
```

Strong Read-After-Write with Quorum:

```
Write quorum (W=3 of 5):
  Client A: PUT user-profile.json
  → S3 writes to Nodes 1, 2, 3, 4, 5 (all 5)
  → Waits for 3 acknowledgments (quorum)
  → Returns "200 OK" only after quorum confirmed

  t=0ms:  Write starts
  t=10ms: Nodes 1, 2, 3 acknowledge → quorum reached
  t=10ms: "200 OK" returned to client
  t=15ms: Nodes 4, 5 also complete (async, after ack)

Read quorum (R=3 of 5):
  Client B: GET user-profile.json
  → Reads from 3 nodes
  → Returns highest-version response
  → Guaranteed: at least 1 of the 3 readers saw the write
     (because W + R = 3 + 3 = 6 > 5 = N)
     ↑ This overlap guarantees consistency!

Read-after-write math:
  N = 5 replicas
  W = 3 (write quorum)
  R = 3 (read quorum)
  W + R > N: 3 + 3 = 6 > 5 → STRONG CONSISTENCY GUARANTEED

  W + R ≤ N: e.g., W=1, R=1, N=5 → EVENTUAL CONSISTENCY
  (Used by S3 Standard for performance, DynamoDB by default)

AWS S3 actual behavior (since Dec 2020):
  S3 now provides strong read-after-write consistency for all operations.
  Implementation: Internal version-based routing. Details not public.
```

---

3. Erasure Coding vs Replication

3.1 Storage Amplification — Worked Numeric Example

3x Replication (Simple):

```
Original data: 1 GB file

Storage layout:
  Node 1: [1 GB copy — full file]
  Node 2: [1 GB copy — full file]
  Node 3: [1 GB copy — full file]

Total storage consumed: 3 GB
Storage overhead:        (3 - 1) / 1 = 200% overhead
Storage efficiency:      1/3 = 33.3%

Durability: Can lose any 2 of 3 nodes and still recover.

Read performance: Any single node can serve the full file.
                  3 replicas = can serve 3 concurrent reads at full speed.

Write performance: Must write to all 3 nodes.
                   Latency = slowest of the 3 writes (tail latency).
```

Reed-Solomon Erasure Coding (6+3) — Worked Example:

```
Original data: 6 GB file (for clean math)

Step 1: Split into 6 equal data shards:
  D1: 1 GB  (bytes 0–1GB)
  D2: 1 GB  (bytes 1–2GB)
  D3: 1 GB  (bytes 2–3GB)
  D4: 1 GB  (bytes 3–4GB)
  D5: 1 GB  (bytes 4–5GB)
  D6: 1 GB  (bytes 5–6GB)

Step 2: Compute 3 parity shards using Reed-Solomon math:
  P1: 1 GB  (linear combination of D1–D6)
  P2: 1 GB  (different linear combination)
  P3: 1 GB  (different linear combination)

Step 3: Distribute 9 shards across 9 different nodes:
  Node 1: D1 (1GB)   Node 4: D4 (1GB)   Node 7: P1 (1GB)
  Node 2: D2 (1GB)   Node 5: D5 (1GB)   Node 8: P2 (1GB)
  Node 3: D3 (1GB)   Node 6: D6 (1GB)   Node 9: P3 (1GB)

Total storage consumed: 9 GB (for 6 GB of data)
Storage overhead:        (9 - 6) / 6 = 50% overhead
Storage efficiency:      6/9 = 66.7%

Compare: 3x replication = 200% overhead. Erasure coding = 50% overhead.
         Erasure coding uses 4x LESS storage for equivalent durability!

Durability: Can lose ANY 3 of 9 nodes and still recover!
            (Need any 6 of 9 shards to reconstruct)

Recovery example — Node 3 (D3) and Node 7 (P1) fail:
  Surviving shards: D1, D2, D4, D5, D6, P2, P3 (7 of 9)
```

```
  We need any 6. We have 7. Can recover!
  Reconstruct: solve Reed-Solomon linear equations for missing D3 and P1.
```

Storage Cost Comparison at Scale:

```
Scenario: Store 1 Exabyte (1,000 PB) of data

3x Replication:
  Storage needed: 3 EB
  At $20/TB: 3,000,000 TB × $20 = $60,000,000/month

Erasure Coding (6+3):
  Storage needed: 1.5 EB (50% overhead)
  At $20/TB: 1,500,000 TB × $20 = $30,000,000/month

Savings: $30,000,000/month = $360,000,000/year
         This is why AWS, Google, Facebook all use erasure coding for cold
storage.
```

3.2 Reconstruction Performance — The Trade-Off

Replication Rebuild:

```
Node 5 dies (holds 1 TB of data):

Rebuild process:
  1. Detect failure: heartbeat timeout (~90 seconds)
  2. Find surviving replicas: NameNode/MON lookup
  3. Copy 1 TB from Node 2 (surviving replica) to Node 10 (replacement)
  4. At 1 Gbps network: 1 TB / 1 Gbps = 8,000 seconds ≈ 2.2 hours

CPU cost: Minimal (just copy bytes)
Network: 1 TB transferred (from 1 source to 1 destination)
Complexity: Simple file copy
```

Erasure Coding Rebuild (CPU-Bound):

```
Node 5 dies (holds D5 shard, 1 TB):

Rebuild process:
  1. Detect failure: ~90 seconds
  2. Contact surviving nodes: D1, D2, D3, D4, D6, P1, P2, P3
  3. Read from any 6 surviving nodes: 1 TB each = 6 TB READ from network
  4. Apply Reed-Solomon decode: compute missing D5 from 6 shards
  5. Write D5 to replacement node: 1 TB WRITE

CPU cost: Reed-Solomon math on 1 TB of data = minutes of CPU time
Network: 6 TB read + 1 TB write = 7 TB transferred (vs 1 TB for replication!)
Complexity: RS decode math, requires coordinating 6+ nodes
```

```
Rebuild window matters because:
  During rebuild (2+ hours), you have only N-1 nodes.
  If another node fails during rebuild: danger zone!
  Replication: lose N1 and N2 → still have replica 3. Fine.
  EC (6+3): lose N1, N2, N3 simultaneously → data loss possible!
```

3.3 Decision Matrix

Scenario	Best Choice	Reason
Hot objects (>100 reads/day)	Replication	Any replica serves immediately, no decode overhead
Cold archive (<1 read/year)	Erasure coding	4x storage savings justify slow read
Small objects (<1 MB)	Replication	EC parity shards > data size = inefficient
Large objects (>1 GB)	Erasure coding	Parity overhead amortized, huge savings
Low-latency reads required	Replication	Erasure coding needs multi-node read + decode
Rebuild speed critical	Replication	EC rebuild reads 6x more network data
Maximum durability	Either (similar)	Both achieve 11 nines with right parameters

When Each Wins — Real-World Examples:

```
Amazon S3 Standard:
  Uses erasure coding for storage efficiency.
  11 nines durability with ~50% storage overhead.
  Reads: single shard fetch (fast path for sequential reads).

Facebook f4 (cold storage):
  EC with 14+3 configuration (78% efficiency vs 33% for 3x replication).
  Stores photos that are rarely accessed (old uploads).
  Saves ~53% storage cost for 65 billion photos.

Google Colossus (GFS successor):
  Uses Reed-Solomon for cold data.
  Replication for hot data and metadata.
```

```
  Adaptive: hot objects automatically promoted to replication.

HDFS:
  Default: 3x replication (simple, fast rebuild)
  HDFS EC (added in 3.0): RS-6+3 for cold directories
  Configured per-directory: hot dirs replicated, cold dirs EC
```

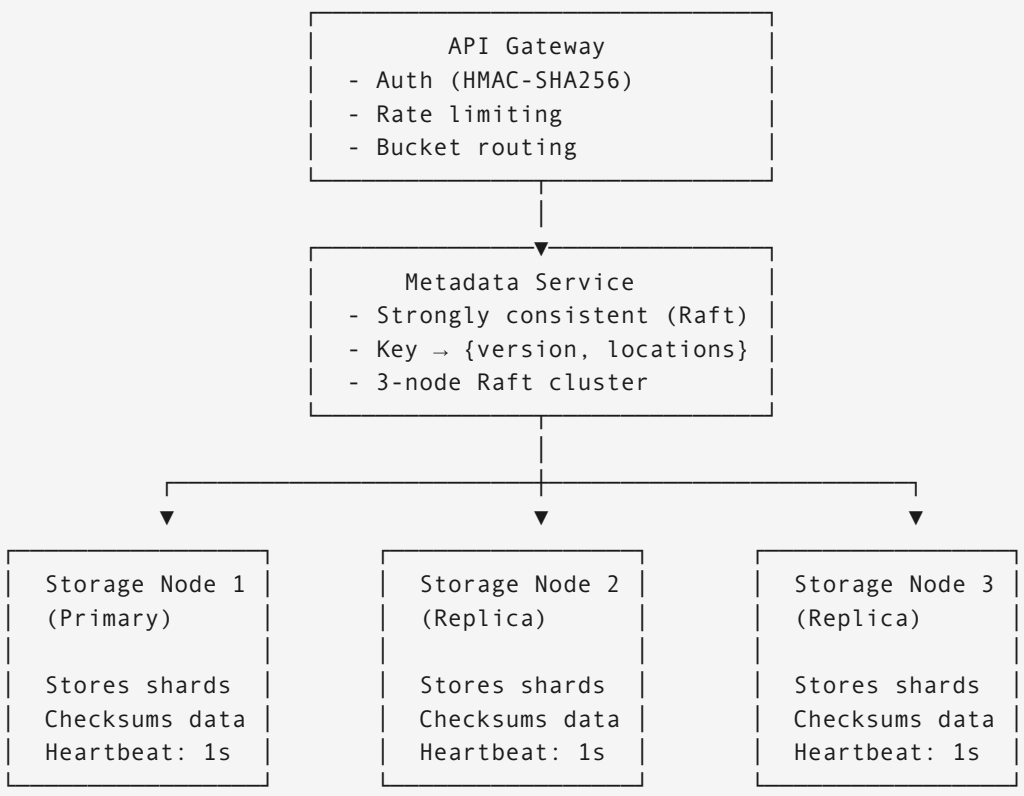
---

4. Interview Question — Extended Solution

Design a distributed storage system that provides S3-like semantics with strong read-after-write consistency. How do you handle write performance with quorum requirements?

Solution Framework

Architecture:



Quorum Write Implementation:

```
async def s3_put(bucket: str, key: str, data: bytes) -> PutResult:
    # Step 1: Generate version ID and checksum
    version_id = uuid4()
    checksum = sha256(data)
```

```
    object_size = len(data)

    # Step 2: Determine replica set via consistent hashing
    replicas = consistent_hash(f"{bucket}/{key}", replication_factor=5)
    quorum_size = len(replicas) // 2 + 1  # 3 of 5

    # Step 3: Write to all replicas in parallel
    async def write_to_replica(node, data, version_id):
        try:
            await node.write(bucket, key, data, version_id, timeout=5.0)
            return node, True
        except (TimeoutError, ConnectionError) as e:
            return node, False

    results = await asyncio.gather(*[
        write_to_replica(r, data, version_id) for r in replicas
    ])

    # Step 4: Check quorum
    successful = [r for r, ok in results if ok]

    if len(successful) >= quorum_size:
        # Step 5: Commit to metadata service (Raft-based, strongly consistent)
        await metadata_service.commit(bucket, key, {
            'version_id': version_id,
            'locations': [r.id for r in successful],
            'checksum': checksum,
            'size': object_size,
            'timestamp': now()
        })

        # Step 6: Background replication to failed nodes
        for failed_node, _ in [(r, ok) for r, ok in results if not ok]:
            asyncio.create_task(background_replicate(failed_node, bucket, key,
version_id))

        return PutResult(version_id=version_id, etag=checksum)

    else:
        # Rollback: delete from successful nodes
        await asyncio.gather(*[r.delete(bucket, key, version_id) for r in
successful])
        raise InsufficientReplicasError(
            f"Only {len(successful)} of {quorum_size} required replicas succeeded"
        )
```

Read-After-Write Guarantee:

```
async def s3_get(bucket: str, key: str) -> GetResult:
    # Step 1: Read current version from metadata (strongly consistent via Raft)
    metadata = await metadata_service.get(bucket, key)
    # metadata.version_id is the latest committed version
    # metadata.locations has all nodes that have this version
```

```
    # Step 2: Read from nearest replica that has the committed version
    sorted_replicas = sort_by_latency(metadata.locations)

    for replica in sorted_replicas:
        try:
            data = await replica.read(bucket, key, version_id=metadata.version_id)
            # Verify integrity
            if sha256(data) == metadata.checksum:
                return GetResult(data=data, version_id=metadata.version_id)
        except (TimeoutError, ChecksumError):
            continue  # Try next replica

    raise InsufficientReplicasError("No healthy replica found")

# WHY this guarantees read-after-write:
# - Write only returns "200 OK" AFTER metadata is committed to Raft
# - Raft commit is strongly consistent (all reads see all committed writes)
# - Read FIRST checks metadata (what is the latest committed version?)
# - Read THEN fetches that exact version from storage
# - Even if replicas lag, metadata tells us what "latest" means
# - We fetch by version_id, not "latest on disk"
```

Performance Optimization Strategies:

```
Problem: Quorum write adds latency (wait for 3 of 5 nodes)
         Latency = slowest of the 3 fastest nodes (tail latency)

Optimization 1 — Hedged Requests:
  After 50ms, send request to 4th node too (don't wait for 3rd to timeout)
  Accept first 3 responses (whichever nodes are fastest)
  Cancel remaining requests
  Effect: Tail latency p99 drops from 200ms to 50ms

Optimization 2 — Write Pipeline:
  Instead of: Client → Node1 → (wait) → Node2 → (wait) → Node3
  Use: Client → Node1 (who pipelines to Node2 and Node3 simultaneously)
  Saves: 2 × RTT client-to-node latency

Optimization 3 — Regional Quorum:
  5 replicas across 3 regions: us-east (2), us-west (2), eu-west (1)
  Quorum satisfied by 2 us-east + 1 us-west = 3 of 5
  Local region quorum: usually satisfiable with 1 RTT
  Cross-region quorum: only needed for global consistency guarantee

Durability vs Performance:
  Strong (W=3, R=3, N=5): 15ms writes, 10ms reads
  Eventual (W=1, R=1, N=5): 3ms writes, 3ms reads
  Most workloads can use eventual (user sessions, analytics)
  Some workloads MUST use strong (financial records, inventory)
```

Key Insight: Strong read-after-write consistency requires that the metadata layer (what is the latest version?) be strongly consistent, even if the data layer (actual bytes) uses eventual replication. By separating metadata consistency (Raft/Paxos) from data replication (quorum with background repair), you get both correctness and performance.

---

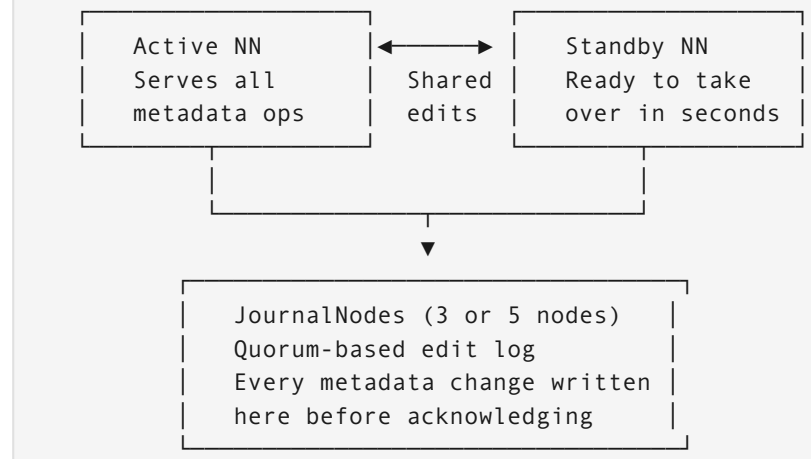
5. Additional Interview Questions

Q2: HDFS NameNode is a single point of failure. How do you design HA for it?

```
Problem: NameNode holds all metadata in RAM.
         Single NameNode crash = entire HDFS unavailable.
         Recovering from edit log can take 30+ minutes at scale.

Solution — HA NameNode (HDFS 2.0+):

Architecture:
```




```
How it works:
  1. Active NN writes every edit to JournalNodes (quorum: 2 of 3)
  2. Standby NN reads from JournalNodes continuously (stays in sync)
  3. DataNodes send block reports to BOTH Active and Standby
  4. Standby maintains full metadata in RAM (ready to serve immediately)

Failover:
  Active NN dies → Standby promoted in ~30 seconds
  No edit log replay needed (already in sync)
  ZooKeeper detects failure + manages leader election

Before HA (HDFS 1.0):
  NameNode crash → read edit log from disk → replay 100M operations
  Recovery time: 30–60 minutes → unacceptable for production
```

Q3: You're designing a storage tier for ML training data. 10 PB of training datasets, read once per training run, write once. Optimize for cost and throughput. Which storage architecture?

```
Requirements analysis:
  - Write once, read ~monthly (cold-ish access pattern)
  - Read pattern: sequential (ML reads files linearly)
  - Throughput: critical (GPU training waits for data = $$$)
  - Cost: 10 PB is expensive, must optimize
  - Durability: important (losing training data is catastrophic)

Architecture decision:

Storage format: Large files (avoid small file problem)
  ML datasets often contain millions of small images (32KB each)
  Naive storage: 1M files × overhead per file = NameNode memory exhaustion
  Solution: Pack into large container files
    TFRecord: TensorFlow's sequential binary format
    WebDataset: TAR archives of samples (POSIX-compatible)
    Parquet: For tabular data (columnar, excellent compression)

  Pack 10,000 images into one TFRecord file:
    1M images / 10,000 per file = 100 files total
    100 files: trivially managed by any storage system

Erasure coding: YES (cold data, large files)
  RS 14+2 (Facebook's choice for cold storage):
    14 data shards + 2 parity = 87.5% efficiency
    vs 33% for 3x replication
    Saves 10 PB × (1 - 0.875/0.333) = saves 7.4 PB of storage

Placement: Cluster co-location
  Training GPU cluster and storage in same datacenter
  Avoid cross-DC bandwidth cost (enormous at PB scale)
  10 PB at 100 Gbps to GPUs = 800 seconds/epoch ← acceptable

Caching layer (critical for throughput):
  Problem: Cold storage = slow reads (EC decode + network)
  Solution: NVMe cache layer between storage and GPUs
    NVIDIA DALI (Data Loading Library): prefetch + cache
    Cache: top 10% of training data in NVMe = dramatically reduces stalls

  With cache:      GPU utilization 95% (rarely waits for data)
  Without cache:   GPU utilization 60% (often stalls waiting for disk)

Cost estimate at 10 PB:
  Replication (3x): 30 PB × $20/TB/month = $600K/month
  Erasure coding:   11.4 PB × $20/TB/month = $228K/month
  Monthly savings: $372K → $4.4M/year
```